FOUR RUNTIMES, FOUR RESOLUTION PLANES, ONE JOIN

the platforms never share memory or a runtime: only data crosses, and every crossing meets in your code

Red dashed = a security gap at the join.

EACH PLATFORM RESOLVES WHO AND WHAT ON ITS OWN TERMS

Salesforce

RUNTIME
Java: Apex, SFCC on Rhino, MuleSoft

RESOLVES PEOPLE BY
Data Cloud identity resolution rules

ID KEY
Contact and Account IDs

CONSENT MODEL
Salesforce consent objects

Adobe

RUNTIME
Java (AEM) · PHP (Adobe Commerce)

RESOLVES PEOPLE BY
Experience Platform identity graph

ID KEY
ECID and identity namespaces

CONSENT MODEL
Consent and governance labels

Shopify

RUNTIME
Ruby on Rails · Wasm Functions (Rust, Javy)

RESOLVES PEOPLE BY
Customer accounts

ID KEY
`gid://shopify/Customer/…`

CONSENT MODEL
The Customer Privacy API

Google Merchant · Ads

RUNTIME
Google's, not public · your Python client

RESOLVES PEOPLE BY
Hashed user data (enhanced conversions)

ID KEY
offerId and GTIN

CONSENT MODEL
Consent Mode v2 signals

Every arrow: JSON over HTTPS. Only data crosses.

J1 Consent drops in the mapping
gaps 20 and M3

J2 IDs resolve to different people
a guess becomes a merge

Your join: middleware, iPaaS or agent

the only place the four planes meet, so the gate and the consent field live here

- maps each plane's IDs to the others, with the source of every match
- carries consent as a field on every record it moves
- sends every deletion and opt-out to every plane, and logs each receipt
- validates every payload against the contract, whoever sent it

J3 Deletion and opt-out don't propagate
one plane forgets, three remember

J4 The join keeps copies
gaps 18 and 28

J5 Payloads trusted without the contract

J6 Data moved to a region it shouldn't be in

WHAT CROSSES THE BOUNDARY

QUESTION	ANSWER
Do the platforms share memory or a runtime?	Never. A JVM, a Ruby process, a Wasm module and Google's services only exchange messages.
What crosses?	JSON over HTTPS: records, events, IDs and consent signals.
What does memory safety protect?	Each plane from its own inputs. It stops at the plane's edge.
What protects the crossing?	The data contract (a schema checked on every payload) and the join that enforces it.
Who carries consent and deletion?	Your join. No plane learns about another plane's opt-out or erasure on its own.

SIX GAPS AT THE JOIN — AND THE FIX FOR EACH

J1 Consent drops in the mapping
Fix: Carry consent state as a field on every mapped record; refuse records without it

J2 IDs resolve to different people
Fix: Keep a mapping table with the source of each match; never merge on a guess

J3 Deletion and opt-out don't propagate
Fix: Fan every erasure and opt-out out to every plane; log each plane's receipt

J4 The join keeps copies
Fix: Store IDs and mappings, not payloads; purge on a schedule

J5 Payloads trusted without the contract
Fix: Validate every inbound payload against the schema, whoever sent it

J6 Data moved to a region it shouldn't be in
Fix: Know each plane's regions; route and store by residency, not convenience

Memory safety protects inside each plane. The data contract and the join protect between them.